

Project Description

The overall aim of this challenge is to design a model for a (simplified) **delivery robot**, similar to those employed in restaurants for **food delivery services** [↗\(https://www.reuters.com/article/uk-health-coronavirus-netherlands-idUKKBN2392ME/\)](https://www.reuters.com/article/uk-health-coronavirus-netherlands-idUKKBN2392ME/), package delivery robots such as **Amazon Scout** [↗\(https://www.aboutamazon.com/news/transportation/meet-scout\)](https://www.aboutamazon.com/news/transportation/meet-scout), or tasks like room service and hospital delivery. The model should be capable of operating within a designated environment, with the ability to autonomously navigate between obstacles, and reach the delivery point.

To this end, the groups need to take a step-by-step approach, starting from a simple discrete environment, towards a more realistic problem with continuous state space, using a Reinforcement Learning (RL) framework. The overview lecture at the beginning of the course will provide the fundamental concepts of RL techniques that must be implemented in the first assignment, and in the second (final) assignment, the groups should research their own approach for a delivery robot using state-of-the-art Deep RL methods.

At the end, the problem setting, proposed solution, and performance of the implementation need to be presented in the group final assignment, which includes a detailed report and a functional implementation of the methods. In this assignment, you need to present and justify not only your RL solution but also its applicability, robustness, and performance impact. Consider what information to include and how to effectively convey your ideas in order to convince someone to invest in your project. This does not mean that you are to pretend that your application and product are without flaws; instead, you are expected to analyze and visualize the behavior and benefits of your application and RL solution.

Below, an example scenario is given for a possible company, their product, and what they intend to do with it:

"Delivery Inc. is a producer of self-driving robots that deliver food orders in restaurants in an automated manner. However, their current algorithm is fairly naïve, resulting in it not delivering all food orders and doing so far below optimal. Someone from the data science team came up with the idea to build an environment to simulate and test different algorithms in different circumstances. Their end goal is to have robots that verifiably perform more efficiently than their prior generation."

Simulation Environment

To help you get started, we provide a basic implementation of an environment with a naïve delivery strategy, which is available at [this public GitHub Repository](https://github.com/DataIntelligenceChallenge/2AMC15-2025) [↗\(https://github.com/DataIntelligenceChallenge/2AMC15-2025\)](https://github.com/DataIntelligenceChallenge/2AMC15-2025). This implementation is given as a starting point for your development; however, it is your responsibility to produce a good implementation, an updated environment, and integration. Note that you should not fork the

repository but download it locally or clone it. Your own repositories must be private, and forking makes them public.

To work with this simulation environment, you should **first go to the GitHub repository and follow the steps for Quickstart, then return to this page**. Here, we describe the codebase in depth.

The structure of the repository is as follows:

```

...
2AMC15-2026
|   README.md
|   requirements.txt
|   train.py
|   LICENSE
|
├── agents
|   |   __init__.py
|   |   base_agent.py
|   |   null_agent.py
|   |   random_agent.py
|
├── world
|   |   __init__.py
|   |   environment.py
|   |   grid_creator.py
|   |   grid.py
|   |   grid_creator.py
|   |   gui.py
|   |   helpers.py
|   |   path_visualizer.py
|   |   └── static
|   |       |   style.css
|   |   └── templates
|   |       |   editor.html
|   |       |   grid.html
|   |
|   └── grid_configs
|       |   example_grid.npy
|       |   large_grid.npy
|       |   small_grid.npy
|       |   A1_grid.npy
|       |   super_hard.npy
|
...

```

Training

The Agent and Environment classes are the most important ones, and together with the `train.py` file, they are the ones you will use and develop the most. Their interaction is illustrated in `train.py` and follows the basic Reinforcement Learning training loop, illustrated in the figures below:

```

# Set up the environment
env = Environment(grid, no_gui, sigma=sigma, target_fps=fps,
                  random_seed=random_seed)

# Initialize agent
agent = GreedyAgent()

# Always reset the environment to initial state
obs, info, _ = env.reset()
for _ in trange(iters):

    # Agent takes an action based on the latest observation and info.
    action = agent.take_action(obs, info)

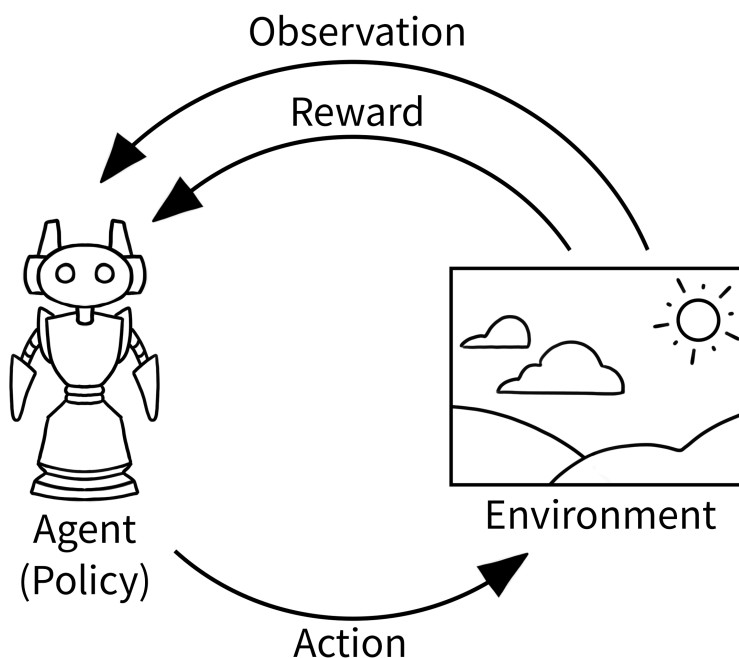
    # The action is performed in the environment
    obs, reward, terminated, info = env.step(action)

    # If the final state is reached, stop.
    if terminated:
        break

    agent.update(obs, reward, info["actual_action"])

# Evaluate the agent
Environment.evaluate_agent(grid, agent, iters, sigma, random_seed=random_seed)

```



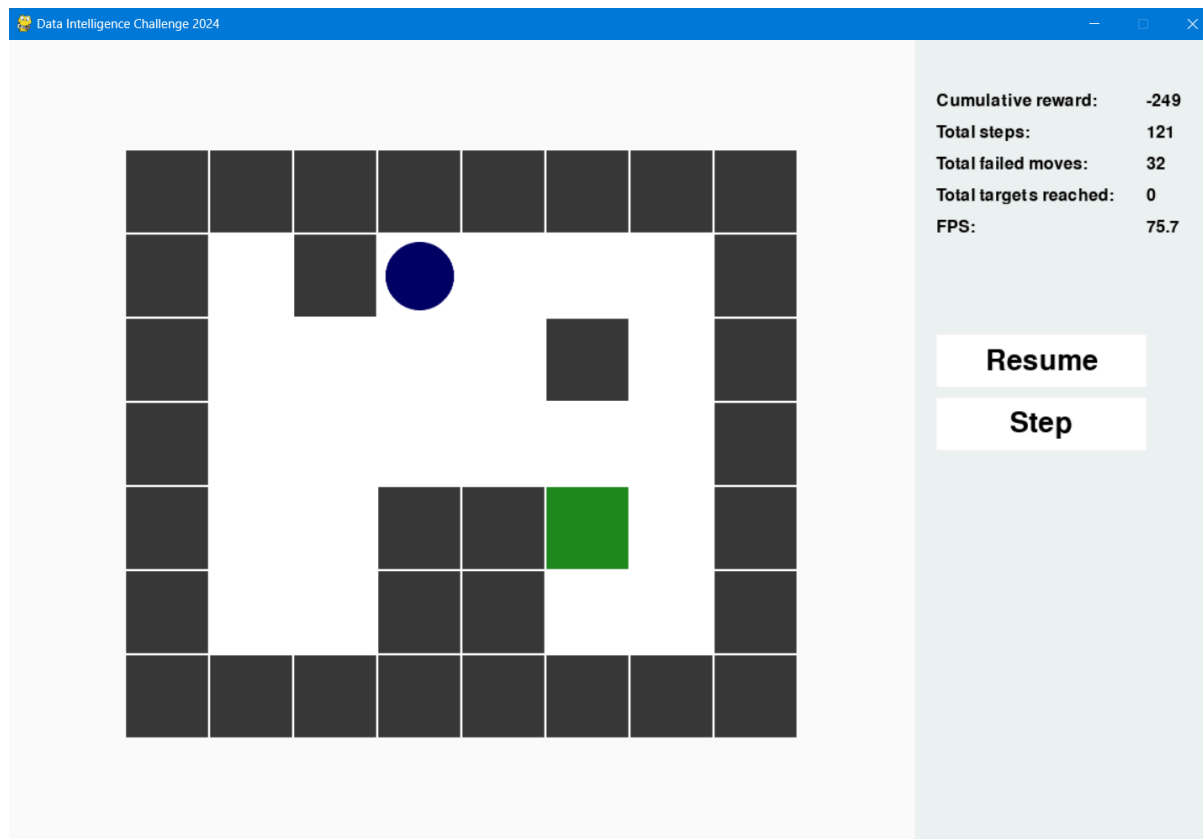
To start, initialize an Environment using a predefined grid and select the Agent you intend to train. Then, reset the environment to its initial state to retrieve the initial observation. In each iteration, the agent selects an action based on the current state of the environment. Following this, the agent interacts with the environment, causing the environment to update itself and provide the next state. If the environment reaches a terminal state, the loop ends; otherwise, the agent adjusts its policy based on the latest observations. The ``obs`` is the state of the grid (a 2D Numpy array

encoding the elements of the world), and ``info`` contains information about the most recent step, and it resets at every step.

You need to develop agents that use the simulation environment to learn to reach their delivery locations as efficiently as possible. In this challenge, you are going to implement an acceptable solution for the problem at-hand. A simple strategy (e.g., random) has been provided in the agents folder for you to get familiar with the overall pipeline. Later, the actions returned by `Agent.take_action(obs, info)` will form an (optimal) policy that is implemented by you.

Environment

To better understand how the agent interacts with the environment, you are provided with a graphical user interface (GUI), which you can activate. This GUI displays various statistics accumulated since the last reset of the environment. **It's important to note** that numbers such as total steps, failed moves, targets reached, or any of their combinations aren't suitable for evaluating your agent's performance. Instead, propose methods that effectively demonstrate your agent's training progress and performance. The GUI's appearance varies depending on the grid configuration, but it should look something like this:



Important parameters when initializing the environment:

- You can turn the graphical user interface off by setting `no_gui` to true; this will speed up the process when you run your experiments.

- You can define the stochasticity of the environment by setting *sigma*. This is the probability that your provided action will correctly be executed (the randomness of the environment).
- You can set the starting position of your agent by passing *start_pos* in the form of a tuple (e.g. 2,3) to indicate the starting position's coordinate. Additionally, if you don't pass any starting position but you have the GUI active you can directly select the position from the GUI.
- You can set your custom reward function by changing it inside the *environment.py* file. A default reward function is already provided.
- You can specify the target speed of the simulation by changing *fps*. Setting it to -1 makes the speed unbound and run as fast as your computer allows.

When resetting the environment, any of the previous parameters can be changed. For example, resetting the environment with `no_gui=True` will deactivate the GUI in the next simulation round: `env.reset(no_gui=True)`.

Inside the Environment class, you will find pieces of code related to the GUI. Since we do not expect you to adjust the GUI, we marked these pieces of code with the comment "GUI specific code", so that you can skip over them.

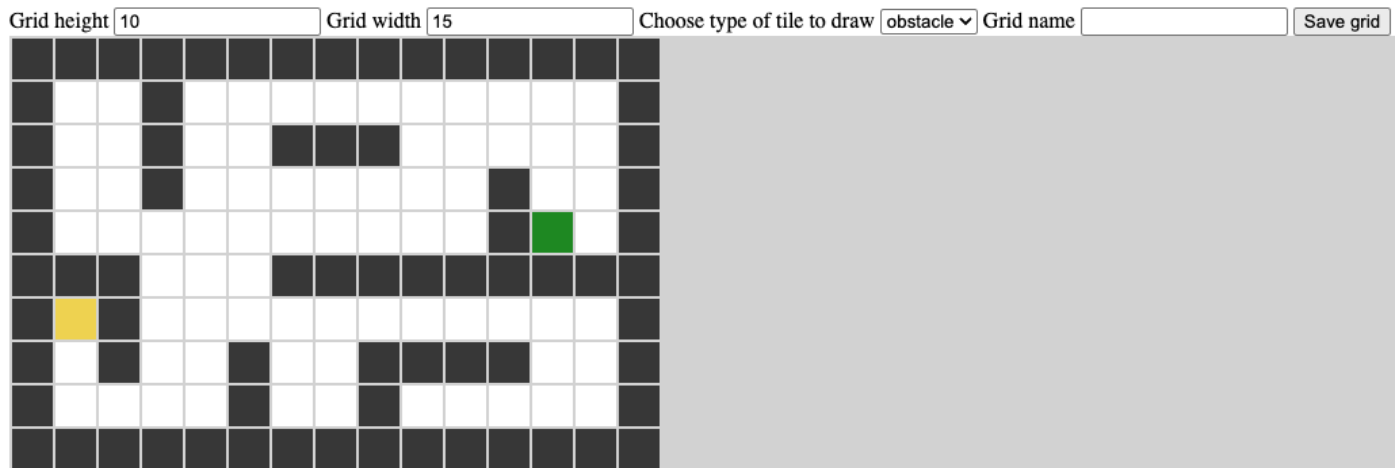
```
# GUI specific code
is_single_step = False
if not self.no_gui:
    start_time = time()
    while self.gui.paused:
        # If the GUI is paused but asking to step, then we step
        if self.gui.step:
            is_single_step = True
            self.gui.step = False
            break
        # Otherwise, we render the current state only
        paused_info = self._reset_info()
        paused_info["agent_moved"] = True
        self.gui.render(self.grid, self.agent_pos, paused_info,
                        0, is_single_step)

# Add stochasticity into the agent action
val = random.random()
if val > self.sigma:
    actual_action = action
else:
    actual_action = random.randint(0, 3)
```

Grid creator

Once you run `grid_creator.py`, it will provide you with a local IP (`http://127.0.0.1:5000`), which can be accessed in your browser by clicking on it. This will give you an intuitive interface to generate

grids according to your preferences. Upon execution, the interface should resemble the following example (depending on the specified parameters):

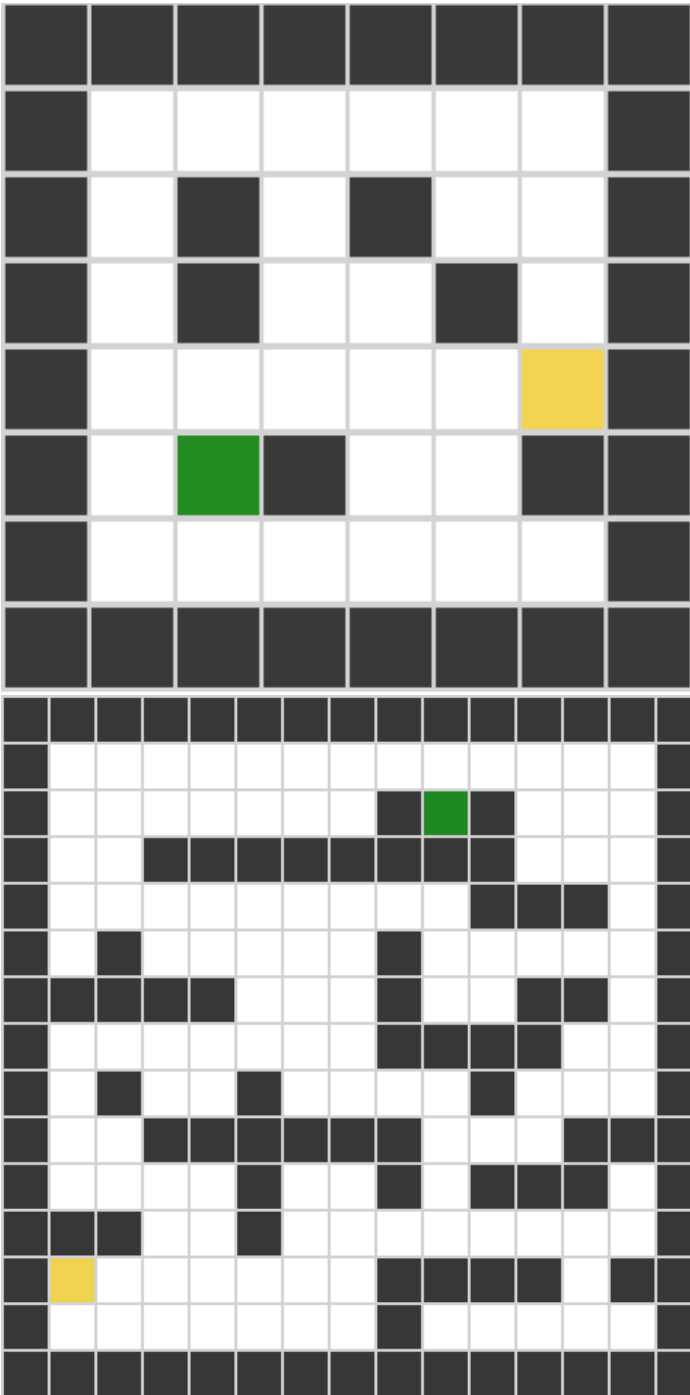


In the above screenshot, some of the settings are already determined, namely, start and target positions, and some obstacles have been added. You can modify them as follows:

- You can change the height and width of the grid by changing the values on the top.
- You can select the type of each tile by choosing the items in the drop-down menu at the top. Any obstacles (shown **black** in the grid) or target (shown **green** in the grid) or start position (shown **yellow** in the grid) have to be chosen by selecting the corresponding item you want to draw from the drop-down menu, and then clicking the grid cell you want to change.
- If you want to remove the item you placed on the grid cell, just click it again. It does not matter which item is selected from the drop-down menu.

The grid you created can be saved by typing a file name on the top-right and clicking on the "save grid" button. Your grid will be saved under `grid_configs/[your_file_name].npy`. The saved file can be loaded as the object `Grid`, which is a 2D Numpy array representation of the world. Each element in the array has a specific value depending on the object it depicts.

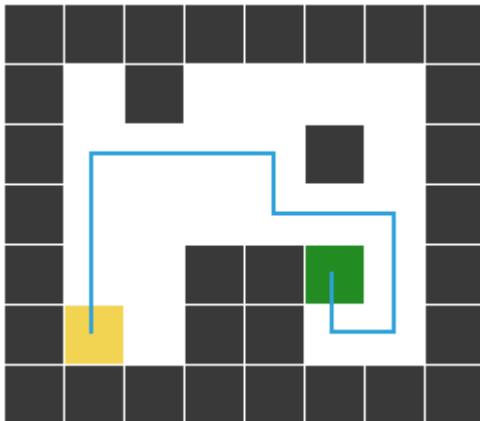
Below you can see examples of different environments from simpler to more complex ones, from which you can get inspired when designing your experiments.



! warning! Make sure your grid is solvable: each target should be reachable.

Evaluation

The environment also includes an `evaluate_agent` method. This can be a useful tool for evaluating how your robot performs. When calling it, it runs the entire agent through the environment with the given number of steps and saves an image in the provided path. It uses the function `visualize_path` from the `path_visualization.py` file. An example of what such an image looks like can be found below.

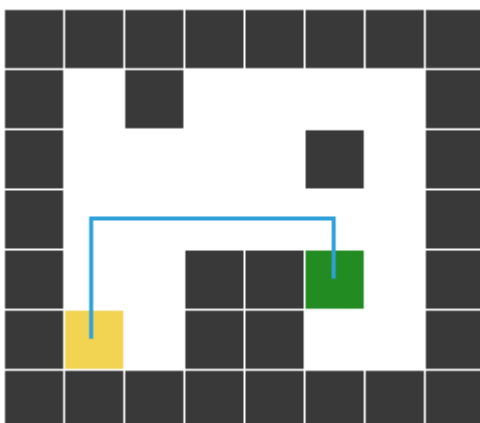


In the above example, we ran the random agent for 100 steps in a room with a few walls and obstacles and a delivery location. The way to interpret the image is as follows:

- The yellow tile shows the starting position of the robot.
- The original black (walls and obstacles) and green (target) tiles remain unchanged and can be seen under the path.
- The light blue lines represent the path that the agent has walked.

In this specific image, we see that our (random) agent reached the target inefficiently; it took a longer path to reach the delivery location. We need to have a smarter approach than moving randomly if we want to reach our targets efficiently.

To get a better idea of what an optimal performance looks like, we trained an agent on the same grid.



In this image, we can see that a trained robot takes the shortest path to the delivery location.

Getting started

Some general notes:

- You are allowed to make changes to the simulation environment. Make sure the changes are useful for improving your agent as best as you can.
- If there are any other files/folders in the codebase that are not described above, we do not expect you to need to modify them.
- Keep in mind the reproducibility aspect! Make sure your work is reproducible.

To get accustomed to the Simulation environment and code from Week 1, take a look at the code, answer the following questions, and try to implement the following:

1. Generate a 13x11 grid with 11 obstacles and 1 target. Save it with the name "test_grid".
2. Run and evaluate the Null and Random agents on a single grid (for example, "test_grid.npy"), with GUI, a stochasticity of the environment of 0.3, 2000 iterations, and a random seed of 100. How does each agent move? What are the results of the evaluation? What happens once the agent reaches the target? Where can the agents move?
3. Train and evaluate the Random agent on 2 grids (for example, "test_grid.npy" and "example_grid.npy") using a single command line.
4. Create a solvable grid and an unsolvable grid using the grid creator. Identify the properties of a solvable grid in contrast to an unsolvable one. What does it mean to be unsolvable? Watch the GUI for the intuitive answer.
5. In the Training section, we were showing how the agent updates after taking a step in the environment. What does it mean that the agent updates based on "actual_move"? What is "actual_move"? What causes it?
6. Try to add at least one diagonal move (top-right) to the Random agent. What do you notice about how moves are made on the grid?
7. How do you know what object each cell of the grid is?
8. What is the difference between the training loop and the evaluation loop?
9. Change the reward function, such that it provides 20 for reaching the target and -7 for stepping on an obstacle.
10. Explain what happens in the step method of the environment (excluding the GUI specific code).